

Oracle PL/SQL – Cursori e Gestione di Più Righe

In PL/SQL abbiamo già visto che `SELECT INTO` è molto comodo quando una query restituisce **una sola riga**. Il problema nasce quando la stessa query può restituire più record: Oracle non può assegnare più righe a una singola variabile scalare e solleva l'eccezione `TOO_MANY_ROWS`.

Questa lezione introduce i **cursori**, cioè il meccanismo con cui Oracle permette di gestire un insieme di righe e di elaborarle una alla volta. L'obiettivo è passare in modo chiaro da una logica "singola riga" a una logica "più righe", usando il dataset `EMPLOYEES` come riferimento principale.



Obiettivi

Capire cursori impliciti, espliciti e Cursor FOR LOOP.



Focus

Elaborare query che restituiscono più righe.



Output

Usare `DBMS_OUTPUT` per osservare il codice in esecuzione.

Il problema iniziale: SELECT INTO su più righe

```
DECLARE
  v_salary employees.salary%TYPE;
BEGIN
  SELECT salary
  INTO v_salary
  FROM employees;
END;
/
```

⚠ Se la tabella `employees` contiene più righe, questo blocco genera `ORA-01422: exact fetch returns more than requested number of rows`. È il segnale che serve un approccio diverso: un cursore.

Definizione e ruolo dei cursori

Un **cursore** è un meccanismo PL/SQL che consente di gestire il risultato di una query. Quando Oracle esegue una `SELECT`, il risultato viene preparato in un'area di memoria associata all'istruzione SQL. Se il risultato contiene più righe, il cursore permette di attraversare questo insieme in modo controllato.

Dall'area di memoria alle righe

Possiamo immaginare il cursore come un "puntatore logico" su un result set. Non leggiamo tutte le righe in una singola variabile: leggiamo una riga alla volta, la copiamo in variabili PL/SQL o in un record, la elaboriamo e poi passiamo alla successiva.

Questo è particolarmente utile quando dobbiamo stampare dati, calcolare valori derivati, verificare condizioni su ogni riga o applicare logiche diverse a dipendenti, ordini, clienti o movimenti contabili.

❗ Il collegamento con la slide precedente è diretto: `SELECT INTO` fallisce se arrivano più righe; il cursore nasce proprio per gestire quel caso in modo ordinato.

Mini tabella concettuale

Approccio

`SELECT INTO`

Cursore

Quando usarlo

Quando la query restituisce esattamente una riga.

Quando la query può restituire più righe.



Cursori impliciti

Creati automaticamente da Oracle per istruzioni SQL eseguite nel codice PL/SQL.



Cursori espliciti

Dichiarati dallo sviluppatore quando serve controllare manualmente il flusso di lettura.

I cursori impliciti

Oracle crea automaticamente un cursore implicito ogni volta che eseguiamo istruzioni SQL come `INSERT`, `UPDATE`, `DELETE` e `SELECT INTO`. Non dobbiamo dichiararlo, aprirlo o chiuderlo: Oracle lo gestisce per noi. Tuttavia possiamo leggere alcune informazioni sull'ultima istruzione SQL eseguita usando gli attributi del cursore implicito, prefissati da `SQL%`.



SQL%FOUND

Restituisce `TRUE` se l'ultima istruzione SQL ha interessato almeno una riga. È utile dopo un `UPDATE` o `DELETE` per sapere se il filtro ha trovato dati.



SQL%ROWCOUNT

Indica il numero di righe elaborate dall'ultima istruzione SQL. È uno degli attributi più usati, soprattutto per confermare quante righe sono state aggiornate.



SQL%NOTFOUND

È il contrario di `SQL%FOUND`. Restituisce `TRUE` se nessuna riga è stata interessata. Può guidare messaggi di controllo o logiche alternative.



SQL%ISOPEN

Per i cursori impliciti è sempre `FALSE` fuori dall'istruzione SQL, perché Oracle apre e chiude automaticamente il cursore.

Esempio concreto dopo un UPDATE

```
BEGIN
  UPDATE employees
  SET salary = salary * 1.1
  WHERE department_id = 50;

  DBMS_OUTPUT.PUT_LINE('Righe aggiornate: ' || SQL%ROWCOUNT);

  IF SQL%FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Aggiornamento eseguito su almeno un dipendente.');
```

```
ELSE
  DBMS_OUTPUT.PUT_LINE('Nessun dipendente trovato per il reparto indicato.');
```

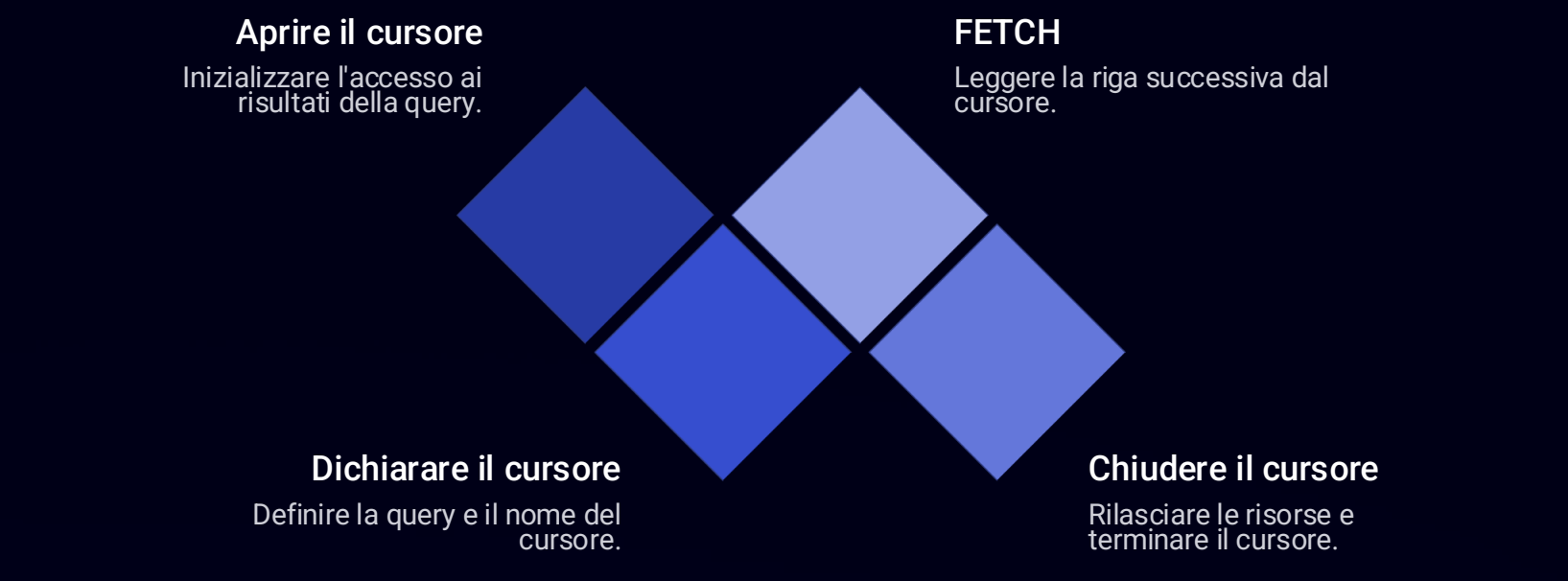
```
END IF;
END;
/
```



Nota: `SQL%ISOPEN` con i cursori impliciti è sempre `FALSE` quando lo controlliamo nel codice PL/SQL, perché non siamo noi a gestire apertura e chiusura.

Quando usare un cursore esplicito

Un cursore esplicito si usa quando vogliamo controllare manualmente la lettura di una query che restituisce più righe. A differenza dei cursori impliciti, qui siamo noi a dichiarare il cursore, aprirlo, recuperare una o più righe e chiuderlo. Questo approccio offre massimo controllo, ma richiede anche più attenzione.



Il ciclo logico è sempre lo stesso: prima definiamo quale query rappresenta il cursore, poi eseguiamo la query con `OPEN`, leggiamo i dati con `FETCH` e infine liberiamo le risorse con `CLOSE`. Un singolo `FETCH` recupera una sola riga: per elaborare tutte le righe servirà un loop, che vedremo nella prossima slide.

Sintassi generale

```
DECLARE
  CURSOR c_emp IS
    SELECT employee_id, first_name, salary
    FROM employees;

  v_id      employees.employee_id%TYPE;
  v_name    employees.first_name%TYPE;
  v_salary  employees.salary%TYPE;
BEGIN
  OPEN c_emp;

  FETCH c_emp
  INTO v_id, v_name, v_salary;

  CLOSE c_emp;
END;
/
```

⚠️ Attenzione: il numero e il tipo delle variabili nel `FETCH` devono corrispondere alle colonne selezionate dal cursore. Se il cursore seleziona tre colonne, il `FETCH INTO` deve avere tre destinazioni compatibili.

OPEN, FETCH, CLOSE nel dettaglio

Per leggere tutte le righe di un cursore esplicito non basta un solo `FETCH`. Serve un `LOOP` che ripeta la lettura finché non ci sono più righe disponibili. La condizione più comune è `EXIT WHEN c_emp%NOTFOUND`, cioè “esci quando l’ultimo `FETCH` non ha trovato una nuova riga”.



Ruolo dei comandi

 OPEN Esegue la query e prepara il result set associato al cursore.	 FETCH Legge una singola riga e la copia nelle variabili indicate.
 CLOSE Chiude il cursore e libera le risorse collegate.	

Esempio completo con LOOP

```
DECLARE
  CURSOR c_emp IS
    SELECT first_name, salary
    FROM employees;

  v_name employees.first_name%TYPE;
  v_salary employees.salary%TYPE;
BEGIN
  OPEN c_emp;

  LOOP
    FETCH c_emp INTO v_name, v_salary;
    EXIT WHEN c_emp%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(v_name || ' - ' || v_salary);
  END LOOP;

  CLOSE c_emp;
END;
/
```

Attributi del cursore esplicito

Attributo	Significato pratico
c_emp%FOUND	TRUE se l’ultimo <code>FETCH</code> ha recuperato una riga.
c_emp%NOTFOUND	TRUE se l’ultimo <code>FETCH</code> non ha recuperato righe; spesso usato per uscire dal loop.
c_emp%ROWCOUNT	Numero di righe recuperate finora dal cursore.
c_emp%ISOPEN	TRUE se il cursore è aperto; utile per controlli difensivi in codice più complesso.

Il modo più pratico: Cursor FOR LOOP

Il **Cursor FOR LOOP** è spesso il modo più semplice, sicuro e leggibile per elaborare tutte le righe restituite da una query. Non dobbiamo scrivere manualmente `OPEN`, `FETCH` e `CLOSE`: Oracle gestisce automaticamente apertura, lettura, uscita dal ciclo e chiusura del cursore.

All'interno del ciclo, la variabile del loop è un **record**: contiene i campi corrispondenti alle colonne selezionate. Se la query seleziona `employee_id`, `first_name` e `salary`, potremo riferirci a `r_emp.employee_id`, `r_emp.first_name` e `r_emp.salary`.

Meno codice

Non serve scrivere la gestione manuale del cursore.

Meno errori

Si riduce il rischio di dimenticare `CLOSE` o creare loop infiniti.

Più leggibilità

Il focus resta sulla logica applicativa da eseguire per ogni riga.

Versione con cursore dichiarato

```
DECLARE
  CURSOR c_emp IS
    SELECT employee_id, first_name, salary
    FROM employees;
BEGIN
  FOR r_emp IN c_emp LOOP
    DBMS_OUTPUT.PUT_LINE(
      r_emp.employee_id || ' - ' ||
      r_emp.first_name  || ' - ' ||
      r_emp.salary
    );
  END LOOP;
END;
/
```

Versione con SELECT nel FOR

```
BEGIN
  FOR r_emp IN (
    SELECT employee_id, first_name, salary
    FROM employees
    WHERE department_id = 50
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(
      r_emp.first_name || ' - ' || r_emp.salary
    );
  END LOOP;
END;
/
```

✅ In molti casi, il **Cursor FOR LOOP** è preferibile al cursore manuale: è più compatto, più chiaro e riduce gli errori tipici nella gestione del ciclo di vita.

Visualizzare i risultati con DBMS_OUTPUT

`DBMS_OUTPUT.PUT_LINE` è una procedura molto usata durante l'apprendimento PL/SQL e nelle fasi di debug. Permette di stampare messaggi testuali e verificare cosa sta succedendo nel codice: quante righe vengono elaborate, quali valori vengono letti, quali rami di un `IF` vengono eseguiti e se un loop sta funzionando come previsto.

Quando è utile

- All'inizio e alla fine di un blocco, per seguire il flusso.
- Dentro un `IF`, per capire quale condizione è vera.
- Dentro un `LOOP`, per vedere ogni iterazione.
- Dentro un cursore, per stampare i valori letti riga per riga.

 In SQL Developer, SQLcl o strumenti simili è necessario attivare il pannello o il comando di output, altrimenti le righe stampate non saranno visibili.

Esempio base

```
BEGIN
  DBMS_OUTPUT.PUT_LINE('Inizio elaborazione');
END;
/
```

Esempio dentro un Cursor FOR LOOP

```
BEGIN
  FOR r_emp IN (
    SELECT first_name, salary
    FROM employees
    WHERE salary > 5000
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Dipendente: ' || r_emp.first_name ||
      ', stipendio: ' || r_emp.salary
    );
  END LOOP;
END;
/
```

 Nota: `DBMS_OUTPUT` è prezioso per apprendimento, test e debug, ma non sostituisce la gestione applicativa dei messaggi, dei log o degli errori in sistemi reali.

Elaborare dati reali con i cursori

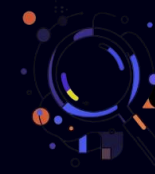
Ora applichiamo i cursori a casi più vicini al lavoro quotidiano su Oracle Database. Il dataset `EMPLOYEES` ci permette di simulare attività comuni: leggere tutti i dipendenti di un reparto, calcolare valori derivati come un bonus teorico, identificare stipendi sopra una soglia e produrre messaggi di controllo leggibili.

Il punto importante non è solo “stampare righe”, ma imparare a ragionare per **record**: ogni iterazione del ciclo rappresenta una riga del risultato SQL, e dentro quella iterazione possiamo applicare calcoli, condizioni, controlli e regole di business.



Calcolo bonus teorico

Per ogni dipendente di un reparto, leggiamo stipendio e nome, poi calcoliamo un valore derivato. È un esempio semplice ma molto vicino a reportistica, simulazioni retributive e controlli HR.



Filtro per stipendio alto

Un cursore può contenere una clausola `WHERE` mirata. In questo modo elaboriamo solo le righe interessanti, riducendo logica PL/SQL inutile e mantenendo il filtro vicino al dato.

Esempio 1: bonus teorico per reparto

```
BEGIN
  FOR r_emp IN (
    SELECT first_name, salary
    FROM employees
    WHERE department_id = 50
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(
      r_emp.first_name ||
      ' - bonus teorico: ' ||
      (r_emp.salary * 0.1)
    );
  END LOOP;
END;
/
```

Qui la query individua i dipendenti del reparto 50. Per ogni riga, PL/SQL calcola il 10% dello stipendio e lo stampa. La formula è dimostrativa, ma il pattern è molto frequente.

Esempio 2: cursore dichiarato per stipendi alti

```
DECLARE
  CURSOR c_high_salary IS
    SELECT first_name, salary
    FROM employees
    WHERE salary > 6000;
BEGIN
  FOR r_emp IN c_high_salary LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Dipendente con stipendio alto: ' ||
      r_emp.first_name ||
      ' (' || r_emp.salary || ')'
    );
  END LOOP;
END;
/
```

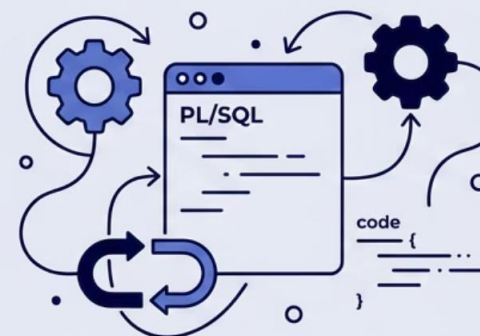
La dichiarazione del cursore dà un nome alla query e rende il codice più leggibile quando la stessa logica deve essere documentata, riutilizzata o discussa in team.

- ✓ Buona pratica: mettere il filtro principale nella `SELECT` quando possibile. PL/SQL dovrebbe elaborare le righe già selezionate correttamente da SQL, non sostituirsi al motore SQL.

Cosa evitare

I cursori sono potenti, ma introducono anche nuove possibilità di errore. Molti problemi nascono da una gestione manuale incompleta: un cursore aperto e non chiuso, un LOOP senza condizione di uscita, oppure un `FETCH INTO` non allineato alla `SELECT`. Per questo è importante riconoscere i rischi e scegliere l'approccio più semplice compatibile con il requisito.

CURSOR FOR LOOP

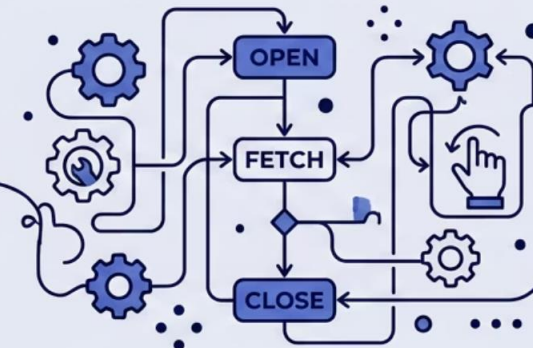


Gestisce **Automaticamente**
OPEN, FETCH, CLOSE

Meno Errori (Meno righe di codice)

Ideale per elaborare
TUTTE le righe

CURSORE MANUALE



OPEN, FETCH, CLOSE
espliciti

Più Controllo sul ciclo

Maggiore Complessità,
più errori possibili

Errori tipici

Dimenticare CLOSE

Nei cursori manuali, dopo `OPEN` e `FETCH` bisogna ricordare `CLOSE`. Lasciare cursori aperti può consumare risorse e rendere il codice fragile.

Scrivere loop infiniti

Un loop senza `EXIT WHEN` `c_emp%NOTFOUND` o senza una condizione corretta può non terminare mai.

Usare `SELECT INTO` per più righe

Se la query può restituire più righe, `SELECT INTO` non è lo strumento giusto: useremo un cursore o un `Cursor FOR LOOP`.

Non allineare `FETCH` e `SELECT`

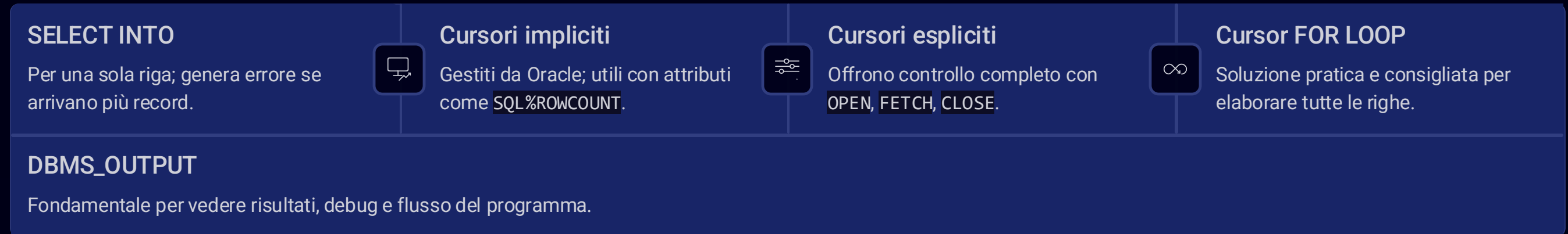
Le variabili del `FETCH INTO` devono corrispondere per numero, ordine e compatibilità di tipo alle colonne selezionate.

Best practice operative

- Preferire `Cursor FOR LOOP` quando l'obiettivo è elaborare tutte le righe.
- Usare nomi chiari: ad esempio `c_emp`, `c_high_salary`, `r_emp`.
- Partire da esempi piccoli prima di introdurre filtri, condizioni e calcoli complessi.
- Verificare sempre la logica del loop con `DBMS_OUTPUT`.
- Tenere la logica di filtro in SQL quando possibile, usando bene `WHERE`, `join` e ordinamenti.

Cosa portiamo a casa

La lezione ci ha accompagnato dal limite di `SELECT INTO` alla gestione strutturata di query multi-riga in PL/SQL. Il messaggio principale è semplice: quando una query restituisce una sola riga, `SELECT INTO` è adeguato; quando le righe sono più di una, dobbiamo usare un cursore o, nella maggior parte dei casi pratici, un `Cursor FOR LOOP`.



Esercitazioni consigliate

1. Leggere e stampare tutti i dipendenti della tabella `employees`.
2. Mostrare nome e stipendio dei dipendenti di un reparto scelto, ad esempio `department_id = 50`.
3. Scrivere un cursore manuale con `OPEN`, `FETCH`, `EXIT WHEN` e `CLOSE`.
4. Riscrivere lo stesso esercizio usando un `Cursor FOR LOOP` e confrontare leggibilità e quantità di codice.

Verso la prossima lezione

I cursori sono una base importante per scrivere codice PL/SQL più strutturato. Nelle prossime lezioni potremo collegare questa logica a oggetti più riutilizzabili e professionali:

- `PROCEDURE`, per incapsulare azioni.
- `FUNCTION`, per restituire valori calcolati.
- `PACKAGE`, per organizzare codice correlato.
- `TRIGGER`, per reagire automaticamente a eventi sul database.

✔ Obiettivo raggiunto: ora sai riconoscere quando `SELECT INTO` non basta, scegliere il tipo di cursore più adatto e usare `DBMS_OUTPUT` per rendere visibile l'elaborazione riga per riga.